

1. Character Checks

Check Vowel

```
return ch=='a' || ch=='e' || ch=='i' || ch=='o' || ch=='u';
```

Alternative

```
return "aeiou".indexOf(ch) != -1;
```

Uppercase + Lowercase

```
return "aeiouAEIOU".indexOf(ch) != -1;
```

Check Digit

```
return ch>='0' && ch<='9';
```

Alternative

```
return Character.isDigit(ch);
```

Check Alphabet

```
return (ch>='a'&&ch<='z') || (ch>='A'&&ch<='Z');
```

Alternative

```
return Character.isLetter(ch);
```

Check Lowercase

```
return ch>='a'&&ch<='z';
```

Alternative

```
return Character.isLowerCase(ch);
```

Check Uppercase

```
return ch>='A'&&ch<='Z';
```

Alternative

```
return Character.isUpperCase(ch);
```

2. Character Conversion

Lower → Upper

```
char upper=(char)(ch-'a'+'A');
```

Alternative

```
Character.toUpperCase(ch);
```

Upper → Lower

```
char lower=(char)(ch-'A'+'a');
```

Alternative

```
Character.toLowerCase(ch);
```

Character to Integer

```
int digit=ch-'0';
```

Integer to Character

```
char ch=(char)(digit+'0');
```

3. String Tricks

Contains Character

```
str.indexOf(ch)!=-1;
```

Alternative

```
str.contains(String.valueOf(ch));
```

Contains Substring

```
str.contains("abc");
```

Reverse String

```
new StringBuilder(str).reverse().toString();
```

Repeat String

```
"ab".repeat(3);
```

Output

```
ababab
```

Convert String to Char Array

```
char[] arr=str.toCharArray();
```

Convert Char Array to String

```
new String(arr);
```

Remove Spaces

```
str.replace(" ","");
```

Trim Spaces

```
str.trim();
```

Split by Space

```
str.split("\\s+");
```

4. Array Tricks

Maximum

```
Math.max(a,b);
```

Minimum

```
Math.min(a,b);
```

Swap

```
int temp=a;  
a=b;  
b=temp;
```

Reverse Array

```
for(int l=0,r=n-1;l<r;l++,r--){  
    int t=arr[l];  
    arr[l]=arr[r];  
    arr[r]=t;  
}
```

Fill Array

```
Arrays.fill(arr,-1);
```

Copy Array

```
Arrays.copyOf(arr,n);
```

Sort Array

```
Arrays.sort(arr);
```

Binary Search

```
Arrays.binarySearch(arr,key);
```

5. Collections

ArrayList to Array

```
list.toArray(new Integer[0]);
```

List Contains

```
list.contains(x);
```

Remove Last

```
list.remove(list.size()-1);
```

Last Element

```
list.get(list.size()-1);
```

Convert Array to List

```
Arrays.asList(arr);
```

6. HashMap Tricks

Increase Frequency

Traditional

```
if(map.containsKey(x))
    map.put(x, map.get(x)+1);
else
    map.put(x, 1);
```

Shortcut

```
map.put(x, map.getOrElse(x, 0) + 1);
```

Decrease Frequency

```
map.put(x, map.get(x) - 1);
```

Remove if Zero

```
if(map.get(x) == 1)
    map.remove(x);
else
    map.put(x, map.get(x) - 1);
```

Check Key

```
map.containsKey(x);
```

Check Value

```
map.containsValue(x);
```

7. HashSet Tricks

Add

```
set.add(x);
```

Exists

```
set.contains(x);
```

Remove

```
set.remove(x);
```

Unique Count

```
set.size();
```

8. Queue

Offer

```
q.offer(x);
```

Remove

```
q.poll();
```

Peek

```
q.peak();
```

Empty

```
q.isEmpty();
```

9. Stack / Deque

Push

```
stack.push(x);
```

Deque

```
dq.push(x);
```

Pop

```
stack.pop();
```

Peek

```
stack.peek();
```

Empty

```
stack.isEmpty();
```

10. PriorityQueue

Min Heap

```
PriorityQueue<Integer> pq=new PriorityQueue<>();
```

Max Heap

```
PriorityQueue<Integer> pq=new PriorityQueue<>(Collections.reverseOrder());
```

Top Element

```
pq.peek();
```

Remove Top

```
pq.poll();
```

11. Bit Manipulation

Even

```
(n&1)==0
```

Odd

```
(n&1)==1
```

Multiply by 2

```
n<<1
```

Divide by 2

```
n>>1
```

Power of Two

```
(n&(n-1))==0
```

Check ith Bit

```
((n>>i)&1)==1
```

Set ith Bit

```
n|(1<<i)
```

Clear ith Bit

```
n&(~(1<<i))
```

Toggle ith Bit

```
n^(1<<i)
```

Count Set Bits

```
Integer.bitCount(n);
```

12. Math Tricks

Absolute

```
Math.abs(x);
```

Square Root

```
Math.sqrt(x);
```

Power

```
Math.pow(a,b);
```

Maximum of Three

```
Math.max(a,Math.max(b,c));
```

Minimum of Three

```
Math.min(a,Math.min(b,c));
```

Ceiling Division

```
(a+b-1)/b
```

13. Loops

Enhanced For

```
for(int x:arr)
```

Reverse Loop

```
for(int i=n-1;i>=0;i--)
```

Two Pointer

```
for(int l=0,r=n-1;l<r;l++,r--)
```

14. Null Safe

```
Objects.equals(a,b);
```

Instead of

```
a.equals(b)
```

15. Boolean Shortcuts

Toggle

```
flag=!flag;
```

Return Boolean Directly

Instead of

```
if(a>b)
    return true;
else
    return false;
```

Simply

```
return a>b;
```

16. Ternary Operator

Instead of

```
if(a>b)
```

```
    max=a;
else
    max=b;
```

Use

```
int max=a>b?a:b;
```

17. Common LeetCode Patterns

Frequency Array

```
int[] freq=new int[26];
```

Index

```
freq[ch-'a']++;
```

ASCII Count

```
int[] freq=new int[128];
```

Prefix Sum

```
prefix[i]=prefix[i-1]+arr[i];
```

Difference Array

```
diff[l]+=x;
diff[r+1]-=x;
```

18. Useful Library Methods

Random

```
Random rand=new Random();
```

Current Time

```
System.currentTimeMillis();
```

String Compare

```
str1.equals(str2);
```

Ignore Case

```
str1.equalsIgnoreCase(str2);
```

Starts With

```
str.startsWith("ab");
```

Ends With

```
str.endsWith("xyz");
```

19. Fast Frequency Count (26 letters)

```
int[] freq=new int[26];  
  
for(char c:s.toCharArray())  
    freq[c-'a']++;
```

20. Common Competitive Programming Idioms

Maximum Value

```
int max=Integer.MIN_VALUE;
```

Minimum Value

```
int min=Integer.MAX_VALUE;
```

Large Number

```
long INF=(long)1e18;
```

Modulo

```
static final int MOD=1_000_000_007;
```

Infinite Loop

```
while(true)
```

Empty String Check

```
str.isEmpty();
```

StringBuilder Append

```
sb.append(x);
```

Convert Integer to String

```
String.valueOf(x);
```

Parse Integer

```
Integer.parseInt(str);
```

Parse Long

```
Long.parseLong(str);
```

Bonus: Frequently Used One-Liners

Task	Shortcut
Check vowel	<code>"aeiou".indexOf(ch) != -1</code>
Frequency update	<code>map.put(x, map.getOrDefault(x, 0) + 1)</code>
Even number	<code>(n & 1) == 0</code>
Odd number	<code>(n & 1) == 1</code>
Ceiling division	<code>(a + b - 1) / b</code>
Reverse string	<code>new StringBuilder(s).reverse().toString()</code>
Char → int	<code>ch - '0'</code>
Int → char	<code>(char)(d + '0')</code>
Lowercase index	<code>ch - 'a'</code>
Uppercase index	<code>ch - 'A'</code>
Count set bits	<code>Integer.bitCount(n)</code>
Toggle boolean	<code>flag = !flag</code>
Return condition	<code>return x < y;</code>

Return condition	<code>return a>0;</code>
Frequency array	<code>freq[ch-'a']++;</code>
Max heap	<code>new PriorityQueue<>(Collections.reverseOrder())</code>
Safe map update	<code>map.put(k,map.getOrDefault(k,0)+1)</code>
Reverse loop	<code>for(int i=n-1;i>=0;i--)</code>
Swap (temp)	<code>int t=a;a=b;b=t;</code>
Last list element	<code>list.get(list.size()-1)</code>
Queue remove	<code>q.poll()</code>

These are the shortcuts and idioms you'll see repeatedly across Java solutions for DSA problems on [LeetCode](#), [HackerRank](#), [Codeforces](#), [CodeChef](#), and [GeeksforGeeks](#).